

Pandas Notes

Introduction to Pandas (Python Library)

Pandas is a powerful open-source Python library used for data analysis, data manipulation, and data cleaning. It is especially useful for working with structured data (tables like Excel or databases).

Why the Name “Pandas”?

Panel data = multi-dimensional data used in statistics and economics.

It provides powerful tools like:

- **DataFrame (table-like structure)**
- **Series (single column)**
- **Data cleaning & transformation**
- **Statistical analysis**
- **analysis**

Origin of Pandas

Pandas was created to solve real-world financial data problems.

- **Pandas was developed by: - Wes McKinney**
- **American software developer & data scientist**
- **Created Pandas while working in finance**
- **Wanted a better tool for analyzing time-series data**

Why Pandas Was Created?

Before Pandas:

- **Excel was slow for large datasets**
- **Databases required complex queries**
- **Python lacked powerful table-based tools**
- **NumPy handled arrays but not labeled data**

After Pandas:

- **Fast data manipulation**
- **Handling missing values**
- **Time-series analysis**
- **Financial data processing**

Timeline of Pandas Development

2008 — Initial Development

- **Wes McKinney started developing Pandas at a hedge fund**
- **Designed mainly for financial modeling**
- **Focused on time-series data**

2009 — Open Source Release

Pandas was released as an open-source project.

- **Developers worldwide could contribute**
- **Rapid improvements began**

2010 — First Public Version

The first official version of Pandas was released.

Key features included:

- **DataFrame object**
- **Series object**
- **Missing data handling**
- **Data alignment**

2011–2014 — Rapid Growth

Pandas became popular in:

- **Data science**
- **Finance**
- **Research**
- **Machine learning**

It became a core part of the Python data ecosystem.

Integration with Scientific Python Stack

Pandas works closely with:

- **NumPy (numerical computing)**
- **Matplotlib (visualization)**
- **SciPy (scientific computing)**

2015 — Industry Adoption

Companies began using Pandas for:

- **Big data analysis**
- **Business intelligence**
- **AI preprocessing**
- **Reporting automation**

2016–Present — Standard Tool for Data Science

Pandas is now:

- **One of the most widely used Python libraries**
- **Essential for data analysts & data scientists**
- **Used in academia and industry worldwide**

Simple Real-Life Workflow

Raw Data → Pandas → Clean Data → Analysis → Insights

Relationship with Python Ecosystem

Pandas sits at the center of data science:

**Raw Data → Pandas → Clean Data → Visualization →
Machine Learning**

Common pipeline:

- **NumPy → numerical operations**
- **Pandas → data manipulation**
- **Matplotlib / Seaborn → visualization**
- **Scikit-learn → machine learning**

Advantages of Pandas

- **Fast and efficient**
- **Easy to learn**
- **Works with large datasets**
- **Integrates with NumPy, Matplotlib, Scikit-learn**
- **Industry standard for data analysis**

Where Pandas is Used in Real Life

Banking: - Fraud detection, transaction analysis

E-Commerce: - Customer behavior, sales trends

Healthcare: - Patient data analysis

Education: - Student performance tracking

Data Science & AI: - Data preprocessing before ML models

Short interview-Ready Summary

- Pandas is an open-source Python library created by Wes McKinney in 2008 for financial data analysis.
 - It provides powerful data structures like Series and DataFrame for handling structured data efficiently.
 - Released as open source in 2009, Pandas quickly became a core tool in data science, analytics, and machine learning due to its ability to clean, manipulate, and analyze large datasets.
-

Installation of Pandas

Pandas is a Python library, so before installing it, you must have Python installed on your system.

Step 1: Install Python

Step 2: Install Pandas Using pip (Recommended)

pip is Python's package manager.

```
pip install pandas
```

Step 3: Verify Pandas Installation

```
import pandas as pd  
print(pd.__version__)
```

Upgrade Pandas

```
pip install --upgrade pandas
```

Uninstall Pandas

```
pip uninstall pandas
```

Check Installed Packages

```
pip list
```

Series in Pandas

A Series in Pandas is a one-dimensional labeled array capable holding data of any type (integer, string, float, Python objects etc.). It can be created from lists, dictionaries etc.

It is similar to:

- **A single column in Excel**
- **A list with labels (index)**
- **A one-dimensional array**

A Pandas Series consists of:

- **Data values**
- **Index (labels)**
- **Data type**

Creating a Series

From a List

import pandas as pd	Output:
data = [10, 20, 30, 40]	0 10
s = pd.Series(data)	1 20
print(s)	2 30
	3 40

With Custom Index

<pre>import pandas as pd data = [10, 20, 30, 40] index1 = ['A', 'B', 'C', 'D'] s = pd.Series(data, index=index1) print(s)</pre>	Output: <pre>A 10 B 20 C 30 D 40</pre>
---	--

Specifying data type

<pre>import pandas as pd data = [10, 20, 30, 40] index1 = ['A', 'B', 'C', 'D'] datatype = 'float' s = pd.Series(data, index=index1, dtype =datatype) print(s)</pre>	Output: <pre>A 10.0 B 20.0 C 30.0 D 40.0</pre>
---	--

Common dtypes in Pandas Series

dtype	Description	Example
-------	-------------	---------

int64	Integer	10, 20
float64	Decimal numbers	3.14
object	String / mixed	"Hello"
bool	True/False	True
datetime64	Date & time	2025-01-01
category	Categorical data	"Low", "High"

From a Dictionary

Keys become index automatically.

import pandas as pd data = {"Math": 85, "Science": 90} s = pd.Series(data) print(s)	Output: Math 85 Science 90
--	---

From a Single Value

Value is repeated for each index.

import pandas as pd index1 = ['A', 'B'] s = pd.Series(10, index=index1) print(s)	Output: A 10.0 B 10.0
---	--

Accessing Elements

Access by Index Label

V_name[index_lable]

Access by Position

V_name[index_value]

Access by Slicing

V_name[start_index : end_index]

Series Attributes

Attribute	Purpose
v_name.values	Returns data
v_name.index	Returns index
v_name.dtype	Data type
v_name.shape	Size of Series
v_name.size	Number of elements

Series Operations

Series supports vectorized operations (applied to all elements).

import pandas as pd	Output:
data = [10, 20]	0 15
s = pd.Series(data)	1 25
print(s+5)	

Mathematical Operations

print(v_name.sum()) # Total

```
print(v_name.mean()) # Average
print(v_name.max()) # Maximum
print(v_name.min()) # Minimum
```

Filtering (Conditional Selection)

<pre>import pandas as pd data = [10, 20] s = pd.Series(data) print(s[s>10])</pre>	Output: 1 20
--	------------------------------

Add Missing Values

<pre>import pandas as pd data = [10, None] s = pd.Series(data) print(s)</pre>	Output: 0 15 1 Nan
---	---

Advantages of Series

- Handles labeled data
- Fast operations
- Supports missing values
- Flexible data types
- Easy data manipulation

Short Interview-Ready Definition

- A Series in Pandas is a one-dimensional labeled array capable of holding data of any type.

- It consists of data values and an associated index.
- It is similar to a single column in a table and forms the basic building block of a DataFrame.

DataFrame in Pandas

A DataFrame is a two-dimensional labeled data structure in Pandas that stores data in rows and columns (like a table).

Why DataFrame is Important?

- The DataFrame is the core object of Pandas.
- Most real-world data analysis tasks use DataFrames.
- Machine Learning also requires DataFrames as input.

A DataFrame consists of:

- Rows (records)
- Columns (fields)
- Index (row labels)
- Column names
- Different data types

It is similar to:

- Excel spreadsheet
- SQL table
- CSV file
- Database table

Real-Life Example

Student Record

Roll No	Name	Maths	Science
---------	------	-------	---------

1	Amit	85	90
2	Neha	78	82
3	Rahul	92	88

Creating a DataFrame

From Dictionary (Most Common)

<pre>data = { "Name": ["Amit", "Neha"], "Math": [85, 78], "Science": [90, 82] } df = pd.DataFrame(data) print(df)</pre>	Output: <table><thead><tr><th></th><th>Name</th><th>Math</th><th>Science</th></tr></thead><tbody><tr><td>0</td><td>Amit</td><td>85</td><td>90</td></tr><tr><td>1</td><td>Neha</td><td>78</td><td>82</td></tr></tbody></table>		Name	Math	Science	0	Amit	85	90	1	Neha	78	82
	Name	Math	Science										
0	Amit	85	90										
1	Neha	78	82										

From List of Lists

<pre>data = [["Amit", 85], ["Neha", 78]] df = pd.DataFrame(data, columns= ["Name", "Math"]) print(df)</pre>	Output: <table><thead><tr><th></th><th>Name</th><th>Math</th></tr></thead><tbody><tr><td>0</td><td>Amit</td><td>85</td></tr><tr><td>1</td><td>Neha</td><td>78</td></tr></tbody></table>		Name	Math	0	Amit	85	1	Neha	78
	Name	Math								
0	Amit	85								
1	Neha	78								

With Custom Index

<pre>data = { "Name": ["Amit", "Neha"],</pre>	Output:
---	----------------

<pre>"Math": [85, 78], "Science": [90, 82] } index1 = ["std1","std2"] df = pd.DataFrame(data, index = index1) print(df)</pre>	<table><tr><th></th><th>Name</th><th>Math</th><th>Science</th></tr><tr><td>Std1</td><td>Amit</td><td>85</td><td>90</td></tr><tr><td>Std2</td><td>Neha</td><td>78</td><td>82</td></tr></table>		Name	Math	Science	Std1	Amit	85	90	Std2	Neha	78	82
	Name	Math	Science										
Std1	Amit	85	90										
Std2	Neha	78	82										

DataFrame Attributes

Attribute	Purpose
v_name.shape	Returns (rows, columns)
v_name.columns	Column names
v_name.index	Row labels
v_name.dtypes	Data types
v_name.values	Data as array
v_name.size	Total elements

Viewing Data

v_name.head()	# First 5 rows
v_name.tail()	# Last 5 rows
v_name.info()	# Summary
v_name.describe()	# Statistics

Selecting Data

Select Single Column

```
v_name ["col_name"]
```

Select Multiple Columns

```
v_name [["col_name1", "col_name2"]]
```

Select Row by Index

```
v_name.loc[index_name] # By label
```

```
v_name.iloc[index_value] # By position
```

Adding New Column

```
v_name ["Total"] = v_name ["Maths"] + v_name ["Science"]
```

Deleting Column

```
v_name.drop("col_name", axis=1, inplace=True)
```

or

```
v_name = v_name.drop("col_name", axis=1)
```

Sorting Data

```
v_name.sort_values("col_name", ascending=False)
```

Find Unique value

```
v_name[col_name].unique()
```

Find value Count

```
v_name[col_name].value_count()
```

Handling Missing Values

Real-world data often contains empty cells.

```
v_name.isnull() # Check missing values
```

v_name.dropna() # Remove missing rows

v_name.fillna(0) # Replace missing values

Data Types in DataFrame

Each column can have a different type:

- Integer
- Float
- String
- Boolean
- DateTime

Interview-Ready Definition

- A DataFrame in Pandas is a two-dimensional labeled data structure with rows and columns capable of storing heterogeneous data.
 - It is similar to a table or spreadsheet and is the primary data structure used for data manipulation and analysis in Pandas.
-